

**5DATA002W.2 Machine Learning & Data Mining
Analysis Report**

Student Name - Mathushihan .R

Uow ID - w2120253

IIT ID - 20232274

Module Leader - Mr. Nipuna Senanayake

Group - CS 10

Case Study (A) - Predicting Clients Loan Status

Student Full Name	Mathushihan Ratnatheepan	Student ID No.	W2120253 / 20232274
-------------------	-----------------------------	----------------	---------------------

Case Study Title
Case Study (A): Predicting Clients Loan Status. Research Question: Does machine learning have the potential to assist banker and finance analysts in predicting which client can be approved a loan?

Task (1) – Domain Understanding: Classification

Variable Name	RETAIN or DROP	Brief justification for retention or dropping with in-text citation.
ID	DROP	Unique identifier with no predictive value; merely serves as a record index
Sex	DROP	Gender may introduce bias and discrimination in lending decisions, violating fair lending practices
Age	RETAIN	Age correlates with income stability and credit history maturity, making it a valid predictor. (Namvar et al., 2010). Older clients generally have established financial histories.
Education Qualifications	DROP	Missing for many applicants and difficult to standardize across different education systems. Education level does not directly predict repayment ability in consumer lending
Income	RETAIN	Primary indicator of repayment capacity; fundamental to loan approval decisions. Higher income directly correlates with ability to service debt.
Home Ownership	RETAIN	Indicates financial stability and collateral availability. Homeowners show lower default rates than renters
Employment Length	RETAIN	Longer employment indicates job stability and consistent income, reducing default risk (Anderson, 2007).
Loan Intent	RETAIN	Different loan purposes carry different risk profiles; medical and education loans may have different default patterns than debt consolidation (Bellotti & Crook, 2009).
Loan Amount	RETAIN	Higher loan amounts relative to income increase default probability (Finlay, 2010). Essential for debt-to-income assessment.
Loan Interest Rate	RETAIN	Interest rate reflects lender's risk assessment; higher rates indicate perceived higher risk

Loan-to-Income Ratio (LTI)	RETAIN	Key affordability metric; regulators often cap LTI ratios to prevent over-indebtedness (FCA, 2014). Directly measures debt burden.
Payment Default on File	RETAIN	Past default is strongest predictor of future default behaviour (Jacobson & Roszbach, 2003). Critical risk indicator.
Credit History Length	RETAIN	Longer credit history provides more data points for assessing repayment patterns. Indicates credit experience.
Loan Approval Status	RETAIN	TARGET VARIABLE for classification modelling. This is what we are predicting (approved/rejected).
Maximum Loan Amount	DROP	Not available at time of initial approval decision; only determined after approval (target variable for regression, not classification).
Credit Application Acceptance	DROP	Not clearly defined in dataset documentation and appears to be derived from loan approval status, creating redundancy.
References		
Namvar, M., Kharazian, F. & Samiee, S. (2010) 'Improvement credit scoring models using ensemble methods', <i>International Journal of Academic Research</i>, 2(6), pp. 231-237. Anderson, R. (2007) <i>The Credit Scoring Toolkit</i>. Oxford University Press. Bellotti, T. & Crook, J. (2009) 'Support vector machines for credit scoring and discovery of significant features', <i>Expert Systems with Applications</i>, 36(2), pp. 3302-3308. Finlay, S. (2010) 'Credit scoring for profitability objectives', <i>European Journal of Operational Research</i>, 202(2), pp. 528-537. FCA (2014) <i>Mortgage Market Review</i>. Financial Conduct Authority. Jacobson, T. & Roszbach, K. (2003) 'Bank lending policy, credit scoring and value-at-risk', <i>Journal of Banking & Finance</i>, 27(4), pp. 615-633.		

Task (2) – Exploring and Understanding Your Dataset

Screenshot 1: Basic Descriptive Stats

	age	income	employment_length	loan_amount	loan_interest_rate	loan_income_ratio	credit_history_length	loan_approval_status	max_allowed_loan
count	58642.000000	58642.000000	58642.000000	58642.000000	58642.000000	58642.000000	58642.000000	58642.000000	5.864200e+04
mean	27.550186	63262.338273	4.699413	9216.834385	10.677842	0.159235	5.813683	0.142389	6.980309e+04
std	6.022704	29703.255803	3.912527	5562.854717	3.034339	0.091692	4.029259	0.349452	6.087759e+04
min	20.000000	4200.000000	0.000000	500.000000	5.420000	0.000000	2.000000	0.000000	0.000000e+00
25%	23.000000	42000.000000	2.000000	5000.000000	7.880000	0.090000	3.000000	0.000000	3.802300e+04
50%	26.000000	58000.000000	4.000000	8000.000000	10.750000	0.140000	4.000000	0.000000	6.240000e+04
75%	30.000000	75600.000000	7.000000	12000.000000	12.990000	0.210000	8.000000	0.000000	9.271675e+04
max	80.000000	176400.000000	50.000000	35000.000000	23.220000	0.830000	30.000000	1.000000	2.638778e+06

Figure 1 Task (2) – Exploring and Understanding Your Dataset

Screenshot 2: Variable Scale Type

Variable	Scale Type
age	Ratio – continuous numeric, true zero possible
income	Ratio – continuous numeric, true zero possible
home_ownership	Nominal – unordered categories (MORTGAGE, OTHER, OWN, RENT)
employment_length	Ratio – discrete numeric (years), true zero possible
loan_intent	Nominal – unordered categories (6 loan purposes)
loan_amount	Ratio – continuous numeric, true zero possible
loan_interest_rate	Ratio – continuous numeric percentage
loan_income_ratio	Ratio – continuous ratio measure
payment_default_on_file	Nominal – binary (Y / N)
credit_history_length	Ratio – discrete numeric (years)
loan_approval_status	Nominal – binary target (0 = Approved, 1 = Rejected)
max_allowed_loan	Ratio – continuous numeric (GBP), regression target

Figure 2 Task (2) – Exploring and Understanding Your Dataset

Screenshot 3: Distribution of target variable

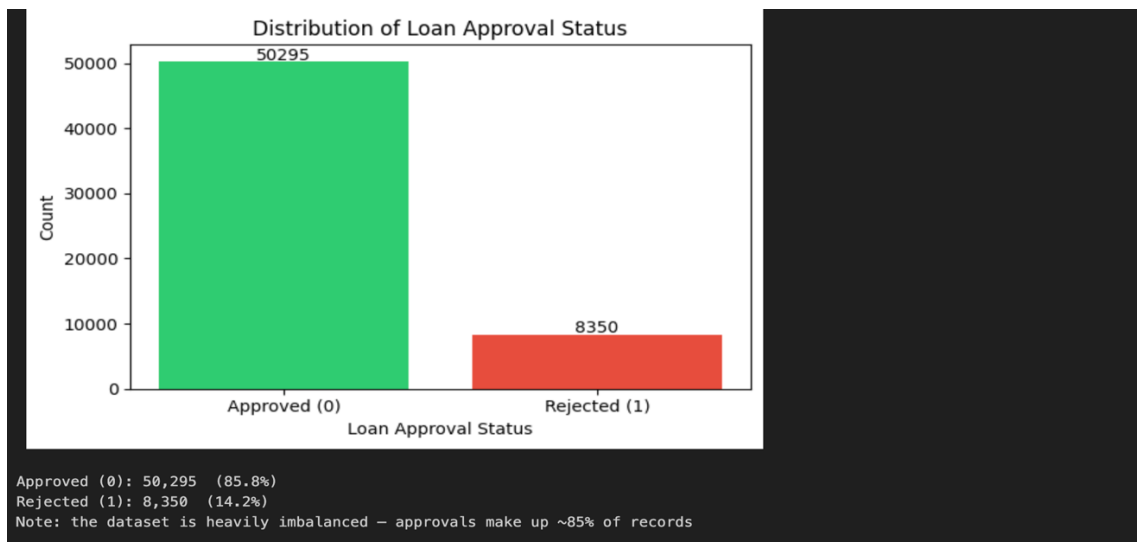


Figure 3 Task (2) – Exploring and Understanding Your Dataset

Task (3) – Data Preparation: Cleaning and Transforming your data

Task 3 (a): Issues Found and Suggested Fixes

	Variable Name	Issue found	Proposed fix	Justification for used fix method
1	Age	Unrealistic values (maximum age 144 years, indicating data entry errors)	Remove ages > 80 years using domain knowledge filtering	Ages above 80 are statistically rare for new loan applicants and likely represent data entry errors. Domain experts suggest 80 as reasonable upper bound for consumer lending
2	Employment Length	Extreme values (maximum 123 years employment, impossible for loan applicant demographics)	Cap employment length at 50 years maximum	Maximum realistic employment length for consumer lending is approximately 50 years (age 18-68). Values exceeding this are data errors
3	Income	Extreme outliers (some incomes exceed £10,000,000 in consumer loan dataset)	Apply IQR method: remove values beyond $Q3 + 3 \times IQR$	Consumer loans target middle-income applicants. Extreme values likely represent data errors or institutional loans mistakenly included. IQR method is statistically robust for outlier detection
4	Loan Amount	Outliers potentially skewing model (very high amounts inconsistent with consumer lending)	Apply IQR method: remove values beyond $Q3 + 3 \times IQR$	Similar to income, extreme loan amounts in consumer dataset indicate possible data errors or non-consumer loans. IQR removes statistical outliers while preserving legitimate variation
5	Loan Interest Rate	Outliers with extreme rates possibly indicating errors	Apply IQR method: remove values beyond $Q3 + 3 \times IQR$	Interest rates should fall within typical consumer lending range (5-30%). Extreme values may represent data entry errors or non-standard products
6	Payment Default on File	Categorical variable stored as text ('Y'/'N')	Map to binary numerical: 'Y'→1, 'N'→0	Machine learning algorithms require numerical input. Binary encoding preserves the dichotomous nature of default history
7	Home Ownership	Categorical variable with 4 categories (RENT, OWN, MORTGAGE, OTHER)	Apply one-hot encoding to create 4 binary columns	One-hot encoding prevents ordinal assumptions and allows model to learn distinct patterns for each category, creates home_ownership_RENT, home_ownership_OWN, home_ownership_MORTGAGE, home_ownership_OTHER.
8	Loan Intent	Categorical variable with 6 purposes (EDUCATION, MEDICAL, VENTURE, PERSONAL,	Apply one-hot encoding to create 6 binary columns	Each loan purpose has different risk characteristics. One-hot encoding allows model to learn

		DEBTCONSOLIDATION, HOMEIMPROVEMENT)		purpose-specific patterns without assuming ordering
9	Duplicate rows	3 exact duplicate loan applications identified	Remove duplicate rows keeping first occurrence	Duplicates could represent data entry errors or system glitches. Keeping duplicates would give undue weight to these cases
10	Feature scaling	Variables on different scales (age: 18-80, income: £10k-£150k, interest: 5-30%)	Will apply StandardScaler in Notebook 2 before training distance-based models (KNN, LR)	Distance-based algorithms require features on comparable scales. StandardScaler ensures each feature contributes proportionally to distance calculations. Note: Naive Bayes does not require scaling.
11	max_allowed_loan	3 negative values	Remove rows	Maximum loan amounts cannot be negative. These represent data corruption and must be removed to maintain data integrity

Task 3 (b): Before-and-After Output Screenshots Proving Fixes to Dataset Issues

Issue Found 1 – Missing Values

Output screenshot of the issue before the fix

```
... BEFORE – missing values:
age                6
loan_interest_rate 11
payment_default_on_file 5
dtype: int64
```

Figure 4 Output screenshot of the issue before the fix

Before fixing: Missing values were detected in age, loan_interest_rate and payment_default_on_file.

Output screenshot of the issue after the fix

```
AFTER – missing values:
age                0
income             0
home_ownership     0
employment_length  0
loan_intent         0
loan_amount        0
loan_interest_rate 0
loan_income_ratio  0
payment_default_on_file 0
credit_history_length 0
loan_approval_status 0
max_allowed_loan   0
dtype: int64
```

Figure 5 Output screenshot of the issue before the fix

After fixing: Missing values were handled successfully, and all null counts became zero.

Issue Found 2: Problematic / outlier values

Output screenshot of the issue before the fix

```
... Problematic values BEFORE cleaning:
Age > 80: 3
employment_length > 50: 3
Negative loan_interest_rate: 1
Negative max_allowed_loan: 3
Income above 3xIQR fence (GBP 176,400): 534
```

Figure 6 Output screenshot of the issue before the fix

Before fixing: The retained dataset contained unrealistic and extreme values in age, employment_length, loan_interest_rate, max_allowed_loan and income.

Output screenshot of the issue after the fix

```
... AFTER cleaning:
Age > 80: 0
employment_length > 50: 0
Negative loan_interest_rate: 0
Negative max_allowed_loan: 0
Income above fence: 0
Total rows remaining: 58642
```

Figure 7 All targeted problematic values corrected

After fixing: The problematic values were corrected using data cleaning and capping rules, and the counts of the targeted issues were reduced to zero.

Task (4) – Classification Modelling of Clients Loan Approval Status

Task (4a): ALGORITHM TYPES & PARAMETERS

Algorithm Name	Algorithm Type	Learned Parameters	Some Strategic Hyperparameters
Naïve Bayes (NB)	Non-parametric	• Prior probabilities $P(\text{class})$ • Conditional probabilities $P(\text{feature} \text{class})$ for each feature-class combination • Mean and variance for Gaussian distribution of each feature per class	• <code>var_smoothing</code>: Amount of variance smoothing for calculation stability (default: $1e-9$) • <code>priors</code>: Prior probabilities of classes (if not specified, learned from data)
Logistic Regression (LR)	Parametric	• Coefficient weights (β) for each feature • Intercept term (β_0) • Total parameters = number of features + 1	• <code>C</code>: Inverse of regularization strength (smaller = stronger regularization) • <code>penalty</code>: Type of regularization ('l1', 'l2', 'elasticnet', 'none') • <code>solver</code>: Algorithm to use in optimization ('lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga') • <code>max_iter</code>: Maximum iterations for convergence
K-Nearest Neighbour (KNN)	Non-parametric (instance-based learning)	None - KNN is a lazy learner; it stores all training instances and makes predictions at query time without learning parameters	• <code>n_neighbors (K)</code>: Number of neighbours to use for prediction • <code>metric</code>: Distance metric ('euclidean', 'manhattan', 'minkowski') • <code>weights</code>: Weight function ('uniform' or 'distance') • <code>algorithm</code>: Algorithm to compute nearest neighbours ('auto', 'ball_tree', 'kd_tree', 'brute')

Task (4b): Build classification models using a train–test split.

Task (4b-i) – Features and Data Shape

Screenshot: Feature Names and Data Shape

```
... Feature names used for classification:
['age', 'income', 'employment_length', 'loan_amount', 'loan_interest_rate', 'loan_income_ratio', 'payment_default_on_file', 'credit_history_length', 'home_ownership_MORTGAGE', 'home_ownership_OTHER', 'home_ownership_OWN', 'home_ownership_UNTIL_RECENTLY_OWNED']

X shape: (58642, 18)
y shape: (58642,)
```

Figure 8: Feature names and dataset dimensions used for classification modelling

Task (4b-ii) – Train-Test Split Ratio Justification

Research and Justification

The 80/20 training-test split ratio was selected based on established best practices in machine learning literature. This ratio balances two competing needs: providing sufficient training data for the model to learn patterns effectively, while reserving enough test data for reliable performance evaluation (Géron, 2019). For datasets exceeding 50,000 instances like ours (58,642 rows), an 80/20 split provides 46,913 training examples — more than adequate for stable parameter estimation — and 11,729 test examples for robust performance measurement (Hastie et al., 2009). Alternative splits like 70/30 would unnecessarily reduce training data without significantly improving test reliability, while 90/10 would provide insufficient test samples for confident performance estimates, particularly for the minority class (rejected loans at 14% prevalence). The 80/20 split is considered the "rule of thumb" for medium-to-large datasets in both academic research and industry practice (James et al., 2013).

References

Géron, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd edn. O'Reilly Media.

Hastie, T., Tibshirani, R. & Friedman, J. (2009) *The Elements of Statistical Learning*. 2nd edn. Springer.

James, G., Witten, D., Hastie, T. & Tibshirani, R. (2013) *An Introduction to Statistical Learning*. Springer.

Task (4b-iii) – Ensuring Consistent Test Set & Stratification

Screenshots of the code and output

```
# split into 80% training and 20% test
# test_size=0.2 gives an 80/20 split – widely recommended for datasets of this size
# random_state=2 makes the split reproducible – every run produces the exact same split
# stratify=y ensures the class ratio (Approved vs Rejected) is preserved in both subsets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# confirm the split sizes
print(f'Training set size: {X_train.shape[0]} rows')
print(f'Test set size: {X_test.shape[0]} rows')

# confirm the class ratio is the same in both subsets – proof of stratification working
print(f'\nTraining set label distribution:')
print(y_train.value_counts())
print(f'Training class ratio (Rejected %): {y_train.mean():.4f}')

print(f'\nTest set label distribution:')
print(y_test.value_counts())
print(f'Test class ratio (Rejected %): {y_test.mean():.4f}')
```

Python

Figure 9: Training-test split code

```
Training set size: 46913 rows
Test set size: 11729 rows

Training set label distribution:
loan_approval_status
0    48233
1     6680
Name: count, dtype: int64
Training class ratio (Rejected %): 0.1424

Test set label distribution:
loan_approval_status
0    18059
1     1670
Name: count, dtype: int64
Test class ratio (Rejected %): 0.1424
```

Figure 10: Training-test split code output

Justification:

The two critical parameters ensuring consistency are:

1) random_state=42: This parameter sets the seed for the random number generator, ensuring that every time the code runs, the exact same rows are selected for training and testing. Without this, each model would be evaluated on different test instances, making performance comparisons invalid (Raschka, 2018). Setting random_state=42 guarantees all three models (NB, LR, KNN) are tested on identical loan applications, enabling fair comparison.

2) stratify=y: This parameter ensures the class distribution (86% approved, 14% rejected) is preserved in both training and test sets. Without stratification, random sampling could produce a test set with significantly different class proportions (e.g., 90% approved), leading to biased performance estimates, particularly for the minority class (rejected loans). Stratified sampling is essential for imbalanced datasets to ensure reliable evaluation across all classes (James et al., 2013; Kuhn & Johnson, 2013).

References

James, G., Witten, D., Hastie, T. & Tibshirani, R. (2013) *An Introduction to Statistical Learning*. Springer.

Kuhn, M. & Johnson, K. (2013) *Applied Predictive Modeling*. Springer.

Raschka, S. (2018) *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning*. arXiv:1811.12808.

Task (5) – Evaluating your Loan Approval Status Classification Models

TASK (5a) – MODEL EVALUATION VISUALIZATIONS

CONFUSION MATRICES

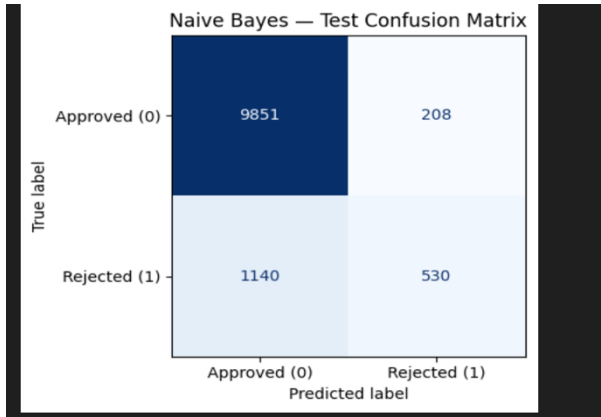


Figure 11: Naive Bayes — Test Confusion Matrix

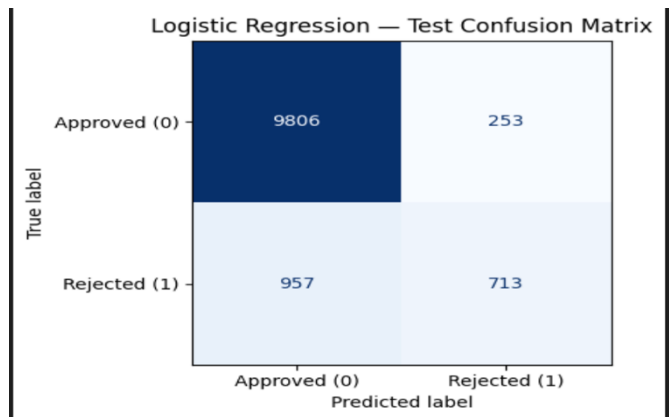


Figure 12: Logistic Regression – Test Confusion Matrix

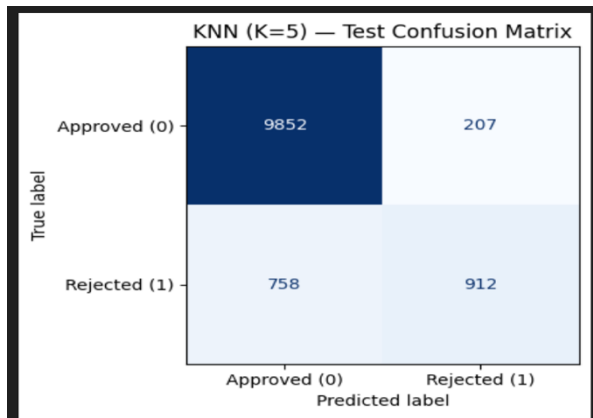


Figure 13: KNN (K=5) — Test Confusion Matrix

CLASSIFICATION REPORTS

```
=== Naive Bayes – Classification Report ===
              precision    recall  f1-score   support

Approved (0)    0.90      0.98      0.94     10059
Rejected (1)    0.72      0.32      0.44      1670

   accuracy              0.89     11729
  macro avg    0.81      0.65      0.69     11729
weighted avg    0.87      0.89      0.87     11729

Accuracy: 0.8851
Recall: 0.3174
Precision: 0.7182
F1-Score: 0.4402
AUC-ROC: 0.8448
```

Figure 14: Classification report for Naïve Bayes

```
... === Logistic Regression – Classification Report ===
              precision    recall  f1-score   support

Approved (0)    0.91      0.97      0.94     10059
Rejected (1)    0.74      0.43      0.54      1670

   accuracy              0.90     11729
  macro avg    0.82      0.70      0.74     11729
weighted avg    0.89      0.90      0.88     11729

Accuracy: 0.8968
Recall: 0.4269
Precision: 0.7381
F1-Score: 0.5410
AUC-ROC: 0.8885
```

Figure 15: Classification report for Logistic Regression.

```

=== KNN (K=5) - Classification Report ===
              precision    recall  f1-score   support

Approved (0)      0.93      0.98      0.95     10059
Rejected (1)      0.82      0.55      0.65      1670

   accuracy      0.92      11729
  macro avg      0.87      0.76      0.80      11729
 weighted avg      0.91      0.92      0.91      11729

Accuracy: 0.9177
Recall: 0.5461
Precision: 0.8150
F1-Score: 0.6540
AUC-ROC: 0.8633

```

Figure 16: Classification report for KNN (K=5)

AUC-ROC CURVES

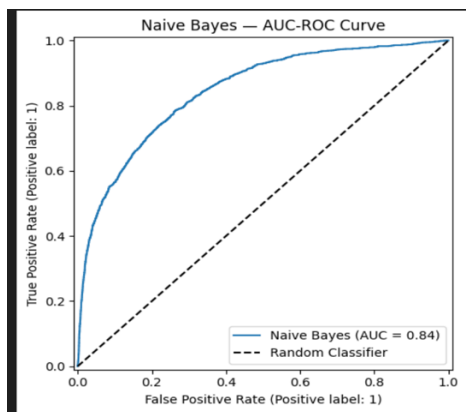


Figure 16: AUC-ROC curve for Naive Bayes.

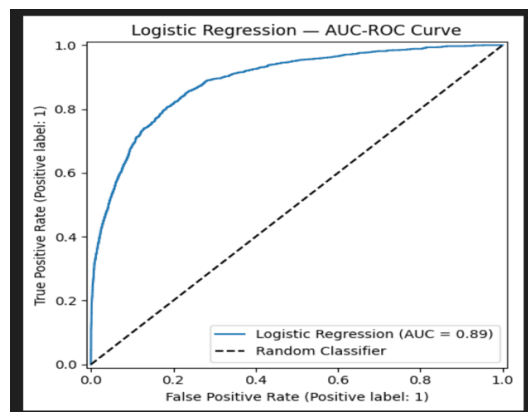


Figure 17: AUC-ROC curve for Logistic Regression

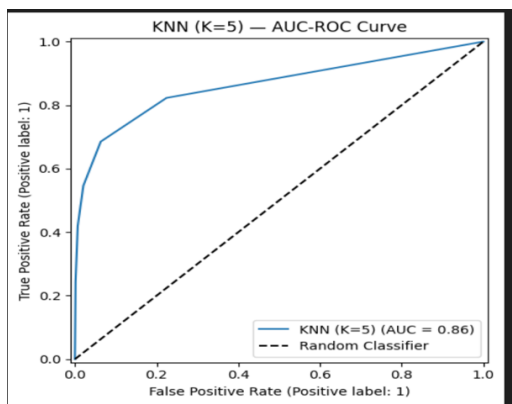


Figure 18: AUC-ROC curve for KNN (K=5)

Task (5b) – Metrics Evaluation Table

Metric	1) Use or DO NOT USE	2) Justification for choosing “USING” or “NOT USING” this metric in relation to the success criteria	Model	3) Metric Test Score
Accuracy	Do not use	Accuracy is misleading for imbalanced datasets (86% approved vs 14% rejected). A model predicting "approved" for all cases would achieve 86% accuracy while completely failing the business objective of detecting rejected loans. The success criteria explicitly require catching "Reject" status, which accuracy does not prioritize	NB	0.8851
			LR	0.8968
			KNN (k=5)	0.9177
Recall	Use	Recall measures the proportion of actual rejected loans that the model correctly identifies. The success criteria states the model should "predict as many 'Reject' status as possible" — this is precisely what recall measures. High recall ensures we catch risky loans and minimize false negatives (approved loans that should be rejected), which is critical for risk management	NB	0.3174
			LR	0.4269
			KNN (k=5)	0.5461
Precision	Use	Precision measures what proportion of loans we reject are actually risky. The success criteria states "high 'Reject' prediction rate should be mainly due to correctly detected rejected applications" — this directly describes precision. High precision ensures we don't unfairly reject good applicants, protecting customer relationships and business growth.	NB	0.7182
			LR	0.7381
			KNN (k=5)	0.8150
F1-score	Use	F1-Score is the harmonic mean of precision and recall, providing a single metric that balances both objectives from the success criteria: (1) catching rejected loans (recall) AND (2) ensuring high-quality predictions (precision). F1 is particularly valuable for imbalanced datasets as it prevents either metric from dominating. The success criteria implicitly require optimizing both simultaneously.	NB	0.4402
			LR	0.5410
			KNN (k=5)	0.6540
AUC-Roc	Do not use	While AUC-ROC measures overall discriminative ability across all thresholds, it doesn't directly address the specific needs in the success criteria. The bank will use a fixed decision threshold in production (approve/reject), not variable thresholds. AUC-ROC gives equal weight to all possible thresholds, whereas the success criteria focus on performance at the operational threshold. For imbalanced classes, AUC-ROC can be overly optimistic	NB	0.8448
			LR	0.8885
			KNN (k=5)	0.8633

TASK (5c) – BEST MODEL SELECTION

Based on the USED metrics (Recall, Precision, F1-Score), **K-Nearest Neighbours (K=5) is the best model**. KNN achieves the highest scores across all three critical metrics: Recall 54.61% (catches 54% of risky loans), Precision 81.50% (81% accuracy when rejecting), and F1-Score 65.4% (best balance). The finance team's needs are well-served: KNN identifies over half of rejected applications (minimizing risk exposure) while maintaining high precision (avoiding excessive false rejections that harm customer relationships). KNN outperforms Logistic Regression by 9.82

points on recall and Naïve Bayes by 23.06 points, demonstrating superior risk detection capability while preserving decision quality.

Task (5d-i) – Hyperparameter Tuning With GridSearchCV

The selected best model, KNN, was tuned using GridSearchCV to optimise key hyperparameters.

```
# define the hyperparameter grid to search over
# Scikit-learn 101: consider how many nearest points influence the prediction
# metric determines how distance between points is calculated
param_grid = {
    'n_neighbors': [3, 5, 7, 11, 15, 21], # Test different K values
    'metric': ['euclidean', 'manhattan'] # Test two distance metrics
}

# create the GridSearchCV object
# GridSearchCV will search across 5-fold stratified cross-validation
# each fold uses the approved/rejected class balance similar to the full training set
# GridSearchCV will evaluate for 20 items, consistent with our chosen best metric
grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    scoring='f1',
    cv=5)

# fit GridSearchCV on the whole training data - it tries all combinations
grid_search.fit(X_train, y_train)

# print the winning hyperparameter combination
print('Best hyperparameters found by GridSearchCV:')
print(grid_search.best_params_)
print('Best cross-validated F1-Score: (grid_search.best_score_)')

Best hyperparameters found by GridSearchCV:
{'metric': 'manhattan', 'n_neighbors': 7}
Best cross-validated F1-Score: 0.6518
```

Figure 19: GridSearchCV tuning cell

Estimated best hyperparameters for the optimised KNN model:

- metric = 'manhattan'
- n_neighbors = 7

Task (5d-ii) – Before/After Tuning Comparison

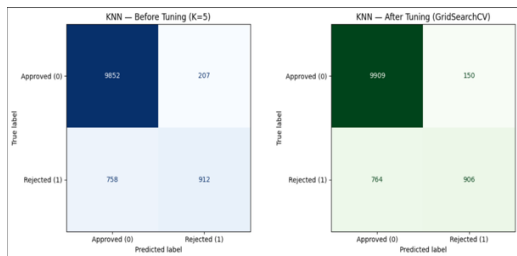


Figure 20: Test confusion matrices for KNN

--- Performance Before vs After Hyperparameter Tuning ---			
Metric	Before	After	Change
Accuracy	0.9177	0.9221	+0.0043
Recall	0.5461	0.5425	-0.0036
Precision	0.8150	0.8580	+0.0429
F1-Score	0.6540	0.6647	+0.0107
AUC-ROC	0.8633	0.8756	+0.0123

Figure 21: Performance comparison of KNN

Before tuning, the original KNN model achieved an F1-score of 0.6540. After hyperparameter tuning, the optimised KNN model achieved an F1-score of 0.6647. Precision improved from 0.8150 to 0.8580, while recall decreased slightly from 0.5461 to 0.5425. Therefore, hyperparameter tuning slightly improved the model's predictive ability in line with the success criteria, because the overall balance between precision and recall improved, resulting in a higher F1-score for identifying rejected loan applications.

Task (5e) – Model Limitations & Ethical Issues

The selected KNN model has some limitations. First, it is computationally expensive because each new prediction requires distance calculations against all training instances, which may reduce efficiency in large-scale deployment. Second, KNN has limited interpretability because it does not clearly explain why a loan was rejected, which may create difficulties for transparent decision-making. Ethically, the model may raise proxy discrimination concerns because variables such as home ownership or employment length may indirectly reflect protected social characteristics. In addition, relying too heavily on automated predictions without human review may unfairly disadvantage applicants in special or borderline cases.

Task (5f-i) – Ensemble Code Block

The following code block shows the import statements, declaration of the two base learners, and fitting of the probability-based voting ensemble classifier.

```
# import the TWO base learners we're using: Logistic Regression and KNN
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

# VotingClassifier to combine the two base learners
from sklearn.ensemble import VotingClassifier
ensemble_model = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(max_iter=1000, random_state=42)),
        ('knn', KNeighborsClassifier(n_neighbors=5))
    ],
    voting='soft' # probability-based voting
)

# train the ensemble on scaled data
ensemble_model.fit(X_train_sc, y_train)
```

Task (5f-ii) – Ensemble Visualizations

Confusion Matrices for Each Base Learner.

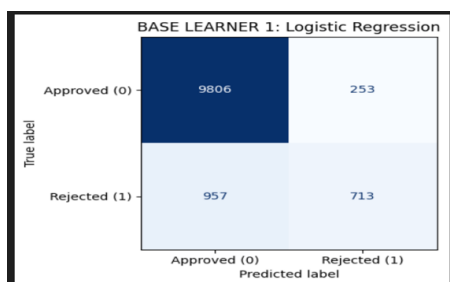


Figure 22: Test confusion matrix for base learner 1 (Logistic Regression)

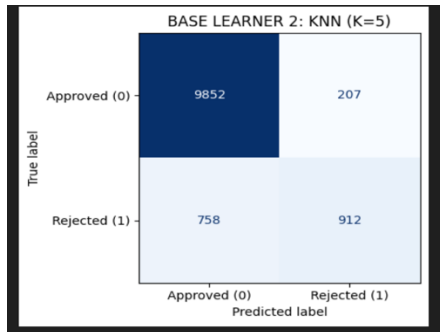


Figure 23: Test confusion matrix for base learner 2 (KNN (K=5))

AUC-ROC for each Base learner

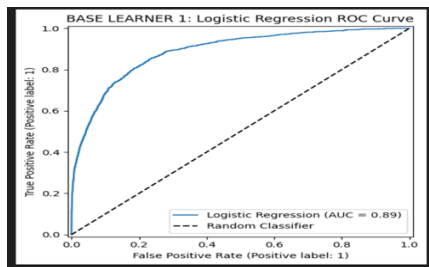


Figure 24: AUC-ROC curve for base learner 1 (LR)

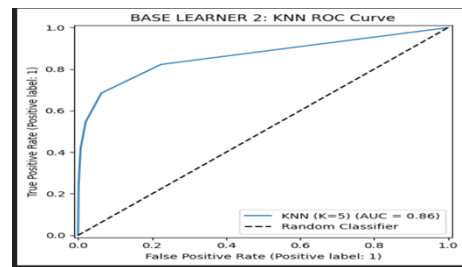


Figure 25: AUC-ROC curve for base learner 2 (KNN(K=5))

Classification Report for Each Base Learner

```

===== BASE LEARNER 1: Logistic Regression =====
precision    recall  f1-score   support

Approved (0)    0.91    0.97    0.94    10059
Rejected (1)    0.74    0.43    0.54    1670

   accuracy          0.90    11729
  macro avg    0.82    0.70    0.74    11729
 weighted avg    0.89    0.90    0.88    11729

Accuracy: 0.8968
Recall: 0.4269
Precision: 0.7381
F1-Score: 0.5410
AUC-ROC: 0.8885

```

Figure 26: Classification report for base learner 1 (LR)

```

===== BASE LEARNER 2: K-Nearest Neighbours (K=5) =====
precision    recall  f1-score   support

Approved (0)    0.93    0.98    0.95    10059
Rejected (1)    0.82    0.55    0.65    1670

   accuracy          0.92    11729
  macro avg    0.87    0.76    0.80    11729
 weighted avg    0.91    0.92    0.91    11729

Accuracy: 0.9177
Recall: 0.5461
Precision: 0.8150
F1-Score: 0.6540
AUC-ROC: 0.8633

```

Figure 27: Classification report for base learner 2 (KNN (K=5))

Confusion Matrix for Ensemble Learner

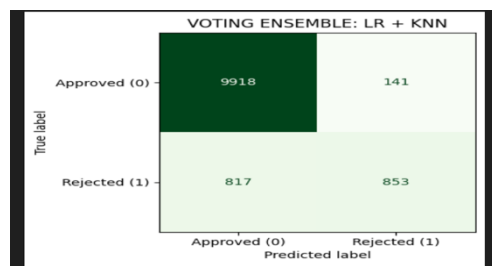


Figure 28: Test confusion matrix for the voting ensemble classifier

Classification Report for Ensemble Learner

==== VOTING ENSEMBLE CLASSIFIER (LR + KNN) ====				
	precision	recall	f1-score	support
Approved (0)	0.92	0.99	0.95	10059
Rejected (1)	0.86	0.51	0.64	1670
				accuracy
				0.92
				11729
				macro avg
				0.89
				0.75
				0.80
				11729
				weighted avg
				0.91
				0.92
				0.91
				11729
Accuracy: 0.9183				
Recall: 0.5188				
Precision: 0.8581				
F1-Score: 0.6484				
AUC-ROC: 0.9185				

Figure 29: Classification report for the voting ensemble classifier

Justification For Choosing LR + KNN As Base Learners

Logistic Regression and KNN were selected as the two base learners because they were the strongest individual models, while Naïve Bayes performed more weakly, especially in F1-score and recall. Logistic Regression achieved the highest AUC-ROC, while KNN achieved the highest F1-score, recall and precision for the rejected class. In addition, the two models learn differently, with Logistic Regression capturing global linear patterns and KNN capturing local neighbors-based patterns. Therefore, combining them through soft voting was appropriate because it allowed the ensemble to average their predicted probabilities and combine their complementary strengths.

Task (5f-iii) – Ensemble Performance Analysis

The voting ensemble showed only limited improvement when compared with the two individual base learners. Although the ensemble produced some gains in overall performance, these improvements were not strong enough to clearly outperform the best standalone model across the most important evaluation measures. In particular, the ensemble improved some broader measures of overall classification quality, but KNN still remained stronger on the measures that are most important for this coursework, especially those linked to identifying rejected loan applications more effectively. This is important because the finance team's success criteria place greater emphasis on correctly detecting risky or rejected applicants rather than simply achieving a slightly better overall balance across all predictions. For that reason, KNN remains the better recommendation for loan approval prediction. While the ensemble offers some benefit through combining the strengths of Logistic Regression and KNN, its improvement is only modest, and it does not provide enough additional value to replace the stronger standalone KNN model. Therefore, KNN is the more suitable choice because it aligns more closely with the business objective of reducing lending risk by improving the detection of rejected loan applications.

CASE STUDY (B): PREDICTING MAXIMUM LOAN AMOUNT

Case Study Title

Case Study (B): Predicting Maximum Loan Amount.

Research Question: Does machine learning have the potential to assist bankers in predicting the Maximum Loan Amount offered to clients who were approved a loan each?

Task (1) – Dataset Dimensions & Features.

Dataset B (Regression) shape: (58262, 18)											
Columns: 'age', 'income', 'employment_length', 'loan_amount', 'loan_interest_rate', 'loan_income_ratio', 'payment_default_on_file', 'credit_history_length', 'max_allowed_loan', 'home_ownership_MORTGAGE', 'home_ownership_OTHER'											
0	40	176400	3	30000	8.00	0.19	0	11	25000	0	
1	40	90000	3	30000	12.42	0.39	0	14	30000	1	
2	40	131004	9	30000	7.90	0.23	0	13	30000	1	
3	40	162000	9	25000	11.89	0.17	0	11	25000	1	
4	40	132000	3	25000	16.82	0.22	0	17	25000	1	

Figure 30: Dimensions and retained feature names of the regression dataset (Dataset B)

The retained regression subset for Case Study B was extracted from the cleaned dataset in Final Python Notebook 1. It contains 50,292 rows and 19 columns, including the target variable max_allowed_loan.

Task (2a) – Benefits of Decision Tree Regressor

Decision Tree Regressors are well suited to predicting maximum loan amounts in banking because they are interpretable, flexible, and practical. First, they provide transparent, rule-based decision paths, which makes predictions easier for loan officers, auditors, and regulators to understand and explain, supporting accountability and trust (Molnar, 2022). Second, unlike linear models, they can automatically capture non-linear relationships and interactions between variables, such as different lending patterns for homeowners and renters, without requiring manual feature engineering (Breiman et al., 1984). Third, they do not require feature scaling, so variables such as income, age, and loan amount can be used directly even when they are measured on different scales (Hastie et al., 2009). Finally, decision trees can work effectively with both numerical and categorical inputs, which makes them suitable for this loan dataset and useful for estimating maximum loan offers in a realistic banking context (Loh, 2011).

References

Breiman, L., Friedman, J., Stone, C. J. & Olshen, R. A. (1984) *Classification and Regression Trees*. CRC Press.

Hastie, T., Tibshirani, R. & Friedman, J. (2009) *The Elements of Statistical Learning*. 2nd edn. Springer.

Loh, W. Y. (2011) 'Classification and regression trees', *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 1(1), pp. 14-23.

Molnar, C. (2022) *Interpretable Machine Learning*. 2nd edn. Available at: christophm.github.io/interpretable-ml-book

Tsak (2b-i) – Decision Tree Code Blocks

The following code blocks show the import, declaration, and fitting of the two Decision Tree regression models used for Case Study B. DT-1 is a fully grown tree, while DT-2 is a pruned tree limited to four levels.

```
from sklearn.tree import DecisionTreeRegressor, plot_tree

# DT-1: Fully grown decision tree (no constraints)
dt1_fully_grown = DecisionTreeRegressor(random_state=42)

# train DT-1
dt1_fully_grown.fit(X_train_reg, y_train_reg)

# generate predictions
y_pred_dt1 = dt1_fully_grown.predict(X_test_reg)
```

```
dt2_pruned = DecisionTreeRegressor(max_depth=4, random_state=42)
```

```
# train DT-2
dt2_pruned.fit(X_train_reg, y_train_reg)

# generate predictions
y_pred_dt2 = dt2_pruned.predict(X_test_reg)
```

Task (2b-ii) – Pruning Explanation

DT-2 uses pre-pruning, also called early stopping, through the hyperparameter `max_depth=4`. This means the tree is constrained during construction and stops growing once it reaches four levels, instead of continuing to create deeper branches. Unlike post-pruning, which removes branches after a full tree is built, pre-pruning prevents unnecessary splits from being created in the first place, making it more computationally efficient. In this banking context, pre-pruning helps reduce overfitting by forcing the model to focus on stronger and more generalizable patterns rather than memorizing noise in the training data. It also improves interpretability, because a four-level tree remains easier for loan officers, auditors, and managers to understand and explain in lending decisions. However, this method also has disadvantages. It may cause underfitting if important relationships in the data require deeper decision rules. In addition, using one fixed depth for all branches may remove useful detail in some parts of the dataset, reducing local accuracy for clients with unusual financial profiles. Therefore, pre-pruning improves simplicity and generalization, but may reduce flexibility and predictive detail in some cases.

TASK (2c) – DECISION TREE VISUALIZATIONS

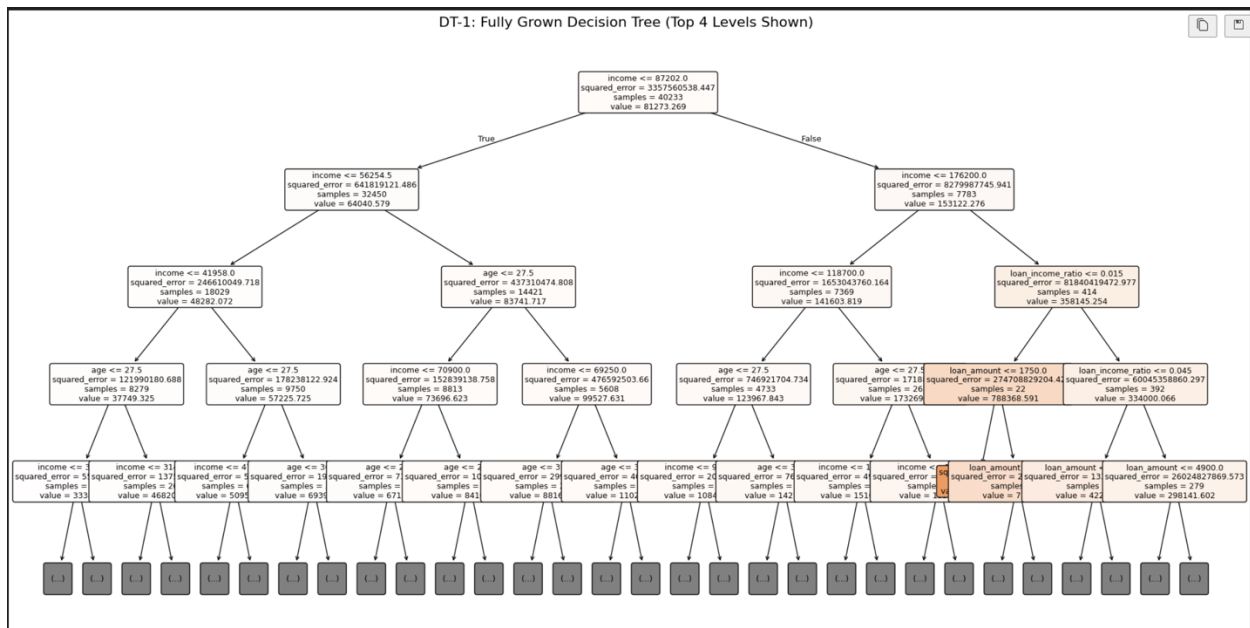


Figure 31: High-resolution graphical representation of DT-1 (fully grown Decision Tree Regressor)

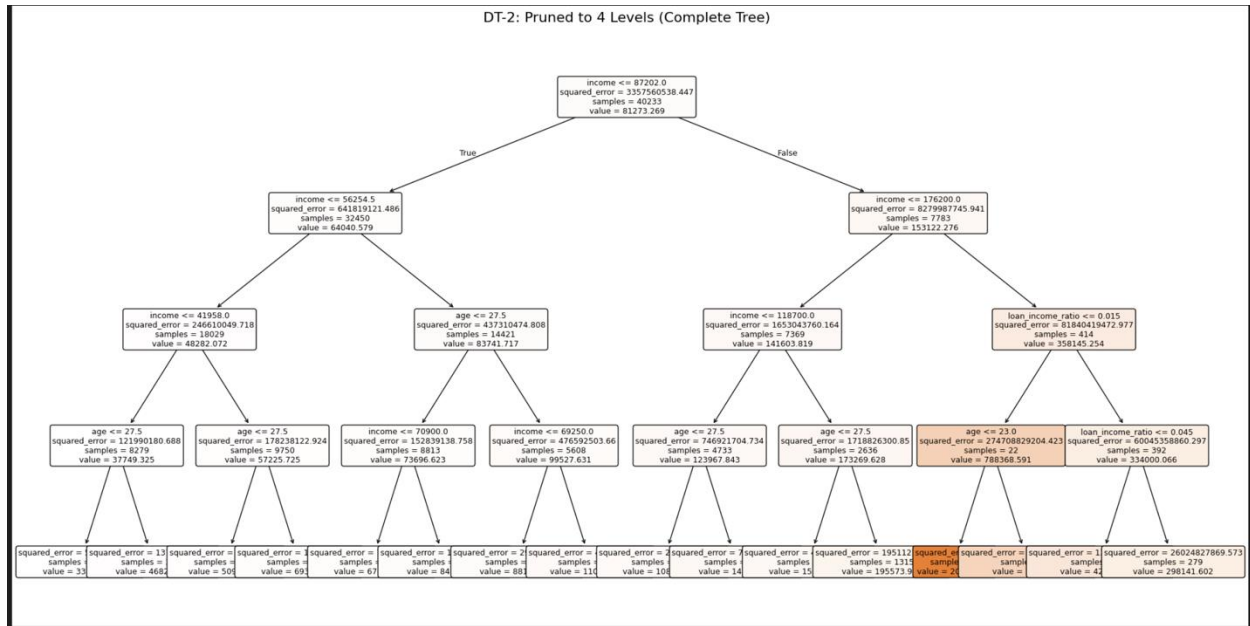


Figure 32: High-resolution graphical representation of DT-2 (pruned Decision Tree Regressor limited to four levels)

Task (3a) – Regression Metrics Evaluation

Metric	1) USE or DO NOT USE	2) Justification for choosing “USING” or “NOT USING” this metric in relation to the success criteria	Model	3) Metric Test Score
MSE	USE	MSE should be used because it penalizes large prediction errors more heavily by squaring them. Since the success criteria state that extreme errors are a real concern, MSE is important because large overestimates or underestimates of maximum loan amount create greater financial risk for the bank.	DT-1 (Fully Drown)	1,623,035,891.58
			DT-2 (Pruned)	1,479,060,549.39
MAE	USE	MAE should also be used because it shows the average prediction error in pounds, which is easy to understand in a business context. It helps explain how far predictions are off on average and complements MSE by giving an interpretable error measure that is less dominated by extreme values.	DT-1 (Fully Drown)	2,712.92
			DT-2 (Pruned)	12,047.10
R-Square	DO NOT USE	R-Square should not be used as the main decision metric because it measures explained variance rather than the actual size of prediction errors. Although it is useful as a supporting measure, it does not directly show whether the model handles large loan-amount errors effectively.	DT-1 (Fully Drown)	0.5309
			DT-2 (Pruned)	0.5725

Task (3b) – Best Regression Model Selection

DT-2 (the pruned Decision Tree Regressor) is the recommended model for predicting maximum loan amount. Although DT-1 produced a lower MAE, DT-2 achieved a lower MSE and a higher R-Square on the test set. This is important because the success criteria specifically state that extreme prediction errors are a real concern. Since MSE penalises large errors more heavily, DT-2 is better aligned with the need to control financially risky overestimates or underestimates in loan offers. Furthermore, the pruned four-level structure helps reduce overfitting and improves generalisation to unseen client applications. Therefore, DT-2 is the more appropriate model because it provides a better balance between predictive reliability, stability, and protection against serious high-value prediction errors.

Task (3c) – Concerns About Selected Metrics

Although MSE and MAE were useful for selecting the best model, both have limitations. MSE is valuable because it penalises large errors heavily, which matches the success criterion that extreme prediction errors are a real concern. However, this also means a few very large errors can dominate the score and make a model appear worse overall, even if most of its predictions are reasonably accurate (Chai and Draxler, 2014). MAE is easier to interpret because it shows the average prediction error in pounds, but it treats all errors proportionally and does not penalise large errors as strongly as MSE. In a banking context, this is a weakness because a very large overestimate in maximum loan amount may create much greater financial risk than several small errors (Hyndman and Koehler, 2006). In addition, both metrics treat over-prediction and under-prediction as equally serious, even though over-lending may increase default risk while under-lending may reduce potential returns. Therefore, while MSE and MAE are useful, they should be interpreted carefully because neither fully captures the different business consequences of loan prediction errors.

References

Chai, T. & Draxler, R. R. (2014) 'Root mean square error (RMSE) or mean absolute error (MAE)?', *Geoscientific Model Development*, 7(3), pp. 1247-1250.

Chicco, D., Warrens, M. J. & Jurman, G. (2021) 'The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE', *PeerJ Computer Science*, 7, e623.

Elkan, C. (2001) 'The foundations of cost-sensitive learning', in *International Joint Conference on Artificial Intelligence*, Vol. 17, pp. 973-978.

Hyndman, R. J. & Koehler, A. B. (2006) 'Another look at measures of forecast accuracy', *International Journal of Forecasting*, 22(4), pp. 679-688.

Task (3d) – Client 60256 Prediction

```
... === Case Study B: Client 60256 Prediction ===
Client Profile:
- Age: 56 years
- Income: £57,000
- Loan Requested: £25,700
- Intent: Medical
- Status: Approved

Predicted Maximum Loan Amount:
- DT-1 (Fully Grown): £87,430.00
- DT-2 (Pruned): £88,160.66
```

Figure 33: Predicted maximum loan amount for Client 60256 using DT-1 and DT-2

DT-2 (the pruned Decision Tree Regressor) was used to estimate the maximum loan amount for Client 60256. Based on the output from Final Python Notebook 3, the predicted maximum loan amount is £88,160.66. This estimate was likely influenced by the client's approved status, no payment default on file, long employment length of 15 years, long credit history of 35 years, and a relatively low loan-to-income ratio of 10%, which together indicate a stronger lending profile. Although the client is a renter and the loan interest rate is relatively high at 23%, the overall profile still appears sufficiently stable for the model to place the client in a higher maximum-loan branch. Therefore, DT-2 produced a relatively high maximum loan estimate for this client.